

GIS805 — Séance 09 / 12 — Les quatre types de tables de faits

Guide de studio (version Markdown). PDF équivalent : docs/lab-guides/GIS805-09_lab.pdf.

En bref

- **Date** : 11 juin 2026
 - **Format** : Travail en autonomie — pas de cours magistral cette semaine
 - **Durée estimée** : 3 h 30 réparties sur 3 à 4 jours
 - **Échéance** : avant le **15 juin 18h00** — la séance S10 (Comité du board) commence le soir même
-

Section 0 — Contexte take-home

Pas de cours cette semaine — voici comment vous organiser

Cette séance n'a pas de cours magistral. Vous travaillez à votre rythme avec ce guide, les diapositives PDF, et la vidéo de synthèse.

Planning suggéré :

Jour	Tâche	Durée
Jeudi 11 juin	Lire les diapositives PDF + regarder la vidéo + make generate && make load	~60 min
Vendredi 12 juin	Exercices 1 et 2 (transaction + snapshot périodique)	~70 min
Samedi 13 juin	Exercices 3 et 4 (accumulating + factless)	~70 min
Dimanche 14 juin	Brief exécutif + arbre de décision + git push	~50 min

Vidéo de synthèse (NotebookLM)

Une vidéo de synthèse de ~6 min couvre la théorie des 4 types de faits, les pièges classiques et le mapping NexaMart. Elle remplace la partie magistrale du cours.

Lien vidéo : Regarder la vidéo de synthèse S09 (~6 min)

Ce lien pointe vers le notebook NotebookLM préparé par l'instructeur — vous n'avez rien à téléverser, écoutez directement.

Ce que vous savez déjà

Depuis S01–S08, vous avez construit : - **fact_sales** — table transactionnelle (S02+) - **dim_customer** avec SCD Type 2 (S05+) et Type 3 (S08) - **bridge_customer_segment** avec pondération (S08)

S09 vous demande d'ajouter 3 nouveaux types de tables de faits à ce modèle existant.

Section 1 — Setup : charger les données S09

Étape 1.1 — Générer et charger

Dans votre terminal, depuis la racine du repo
make generate
make load

Row counts attendus après make load :

Table	Count attendu	Type de fait
fact_sales	~3 800	Transaction (déjà en place)
fact_returns	~250	Transaction (déjà en place)
fact_orders_transaction	~2 000	Transaction (NOUVEAU S09)
fact_daily_inventory	~6 500	Periodic snapshot (NOUVEAU S09)
fact_order_pipeline	~1 200	Accumulating snapshot (NOUVEAU S09)
fact_promo_exposure	~1 500	Factless (NOUVEAU S09)

Note : Ces counts sont des ordres de grandeur — les volumes exacts dépendent du seed de votre équipe. Ne vous inquiétez pas si vos chiffres diffèrent : la validation fiable est **make check** (tous les checks **PASS**) et le fait que chaque table ci-dessus soit **non vide** (l'étape 1.3 affiche vos propres counts).

Étape 1.2 — Vérifier le chargement

`make check`

Résultat attendu : tous les checks **PASS**. Si un check **FAIL**, consultez `docs/TROUBLESHOOTING.md`.

Étape 1.3 — Explorer les tables en bref

```
-- Dans DuckDB : duckdb db/newamart.duckdb
SELECT table_name, estimated_size AS rows
FROM duckdb_tables()
WHERE table_name LIKE 'fact_%'
ORDER BY table_name;
```

Section 2 — Exercice 1 : Table de faits transactionnelle

Concept : Une ligne = un événement ponctuel. Insert-only. Mesures additives sur toutes les dimensions.

Durée estimée : 35 min

2.1 — Vérifier le grain

```
-- Le grain doit être unique : une ligne = une transaction atomique
SELECT
    COUNT(*) AS total_lignes,
    COUNT(DISTINCT transaction_id) AS transactions_distinctes,
    COUNT(DISTINCT transaction_type) AS nb_types_transaction
FROM fact_orders_transaction;
```

Question : `total_lignes` est-il égal à `transactions_distinctes` ? Combien de types de transaction (`sale`, `return`, `exchange`) existe-t-il ? Expliquez dans votre brief pourquoi une table transactionnelle est insert-only.

Réponse attendue : `total_lignes` = `transactions_distinctes` — chaque ligne est une transaction unique identifiée par `transaction_id` (le grain est respecté). Plusieurs `transaction_type` coexistent. Insert-only : on ne modifie ni n'efface jamais une transaction passée.

2.2 — Requête CEO 1 : revenue par catégorie

```
-- Revenue par catégorie de produit
SELECT
    p.category,
    COUNT(*) AS nb_transactions,
    SUM(f.quantity) AS unites_vendues,
```

```

SUM(f.amount)                AS revenue_brut
FROM fact_orders_transaction f
JOIN dim_product p ON p.product_key = f.product_key
GROUP BY 1
ORDER BY revenue_brut DESC;

```

Note : fact_orders_transaction contient amount (montant de la transaction). La table fact_sales contient line_total. Ce sont deux grains différents — fact_orders_transaction inclut les retours et échanges via transaction_type.

2.3 — Requête CEO 2 : top 5 magasins par revenue

```

-- Top 5 magasins par revenue total
SELECT
    ds.store_name,
    ds.region,
    SUM(f.amount)                AS revenue_total,
    SUM(f.quantity)             AS unites_vendues
FROM fact_orders_transaction f
JOIN dim_store ds ON ds.store_key = f.store_key
GROUP BY 1, 2
ORDER BY revenue_total DESC
LIMIT 5;

```

2.4 — Livrable

Sauvegardez vos requêtes dans `sql/fact-types/s09-transaction.sql`.

Un squelette est déjà dans ce fichier avec des `-- TODO` — complétez les sections marquées.

Section 3 — Exercice 2 : Snapshot périodique

Concept : Une ligne = l'état d'une entité à une date. La mesure de stock est **semi-additive** : additive sur produits et magasins, mais pas sur le temps.

Durée estimée : 35 min

3.1 — Vérifier le grain

```

-- Le grain doit être unique : product_key × store_key × snapshot_date
SELECT
    COUNT(*)                    AS total_lignes,
    COUNT(DISTINCT (product_key, store_key, snapshot_date)) AS lignes_uniques
FROM fact_daily_inventory;
-- Attendu : total_lignes = lignes_uniques (grain respecté)

```

3.2 — Démonstration du piège semi-additif

Exécutez ces deux requêtes et notez l'écart.

```

-- FAUX : SUM du stock sur le temps (gonfle les chiffres)
SELECT
    p.category,
    SUM(i.quantity_on_hand) AS stock_cumule_FAUX
FROM fact_daily_inventory i
JOIN dim_product p ON p.product_key = i.product_key
GROUP BY 1
ORDER BY stock_cumule_FAUX DESC;

```

```
-- CORRECT : stock moyen par catégorie (semi-additif traité correctement)
SELECT
    p.category,
    ROUND(AVG(i.quantity_on_hand), 1) AS stock_moyen,
    MAX(i.quantity_on_hand)          AS stock_max,
    MIN(i.quantity_on_hand)          AS stock_min,
    COUNT(DISTINCT i.snapshot_date)  AS nb_jours_observation
FROM fact_daily_inventory i
JOIN dim_product p ON p.product_key = i.product_key
GROUP BY 1
ORDER BY stock_moyen DESC;
```

Dans votre brief : notez le ratio SUM / AVG par catégorie. Ce ratio nb_jours_observation. C'est la preuve que SUM additionne des snapshots répétés.

3.3 — Requête CEO : stock des 5 derniers jours par magasin

```
-- Stock moyen sur les 5 dernières dates de snapshot disponibles
WITH derniers_jours AS (
    SELECT DISTINCT snapshot_date
    FROM fact_daily_inventory
    ORDER BY snapshot_date DESC
    LIMIT 5
)
SELECT
    ds.store_name,
    ds.region,
    ROUND(AVG(i.quantity_on_hand), 1) AS stock_moyen_5j,
    ROUND(AVG(i.days_of_supply), 1)  AS jours_approvisionnement_moyen
FROM fact_daily_inventory i
JOIN derniers_jours d ON d.snapshot_date = i.snapshot_date
JOIN dim_store ds    ON ds.store_key = i.store_key
GROUP BY 1, 2
ORDER BY stock_moyen_5j DESC;
```

3.4 — Livrable

Sauvegardez dans `sql/fact-types/s09-periodic-snapshot.sql` : les deux requêtes (fausse + correcte) + la requête CEO. Le fichier squelette contient des -- TODO.

Section 4 — Exercice 3 : Snapshot accumulant

Concept : Une ligne par processus (ici : une commande), mise à jour au fur et à mesure que les jalons sont franchis. Les dates de jalons non atteints sont NULL. C'est le **seul** type de table de faits qu'on met à jour.

Durée estimée : 35 min

4.1 — Explorer le pipeline

```
-- Vue d'ensemble : statuts et délais
SELECT
    current_status,
    COUNT(*) AS nb_commandes,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (, 1) AS pct_total,
    ROUND(AVG(days_order_to_deliver), 1) AS delai_moyen_jours,
    MIN(days_order_to_deliver) AS delai_min,
    MAX(days_order_to_deliver) AS delai_max
```

```
FROM fact_order_pipeline
GROUP BY 1
ORDER BY nb_commandes DESC;
```

4.2 — Funnel de complétion par jalon

```
-- Quel % des commandes a atteint chaque jalon ?
SELECT
    COUNT(*)                                AS total_commandes,
    SUM(reached_payment::INT)                AS nb_payment,
    SUM(reached_pick::INT)                   AS nb_pick,
    SUM(reached_ship::INT)                   AS nb_ship,
    SUM(reached_delivery::INT)               AS nb_delivery,
    ROUND(100.0 * SUM(reached_payment::INT) / COUNT(*), 1) AS pct_payment,
    ROUND(100.0 * SUM(reached_pick::INT) / COUNT(*), 1) AS pct_pick,
    ROUND(100.0 * SUM(reached_ship::INT) / COUNT(*), 1) AS pct_ship,
    ROUND(100.0 * SUM(reached_delivery::INT) / COUNT(*), 1) AS pct_delivery
FROM fact_order_pipeline;
```

Question pour le brief : Pourquoi `pct_delivery < 100 %` ? Est-ce un problème de données ou un reflet de la réalité opérationnelle ?

Réponse attendue : les commandes avec `delivery_date IS NULL` sont des commandes encore en cours — pas des données manquantes.

4.3 — Analyse des délais entre jalons

```
-- Délai moyen entre chaque étape (seulement pour les jalons franchis)
SELECT
    ROUND(AVG(CAST(payment_date - order_date AS INTEGER)), 1) AS j_order_to_payment,
    ROUND(AVG(CAST(pick_date - payment_date AS INTEGER)), 1) AS j_payment_to_pick,
    ROUND(AVG(CAST(ship_date - pick_date AS INTEGER)), 1) AS j_pick_to_ship,
    ROUND(AVG(CAST(delivery_date - ship_date AS INTEGER)), 1) AS j_ship_to_delivery
FROM fact_order_pipeline
WHERE payment_date IS NOT NULL
    AND pick_date IS NOT NULL
    AND ship_date IS NOT NULL
    AND delivery_date IS NOT NULL;
```

4.4 — Livrable

Sauvegardez dans `sql/fact-types/s09-accumulating.sql`. Le fichier squelette contient les `-- TODO` pour le funnel + analyse des délais.

Section 5 — Exercice 4 : Factless fact table

Concept : Aucune mesure numérique. La présence d'une ligne dans la table **est** l'information. Utile pour mesurer la couverture, l'éligibilité ou la présence. La requête de base est `COUNT(*)`.

Durée estimée : 30 min

5.1 — Explorer la couverture

```
-- Expositions par campagne
SELECT
    campaign_id,
    COUNT(*)                                AS nb_expositions,
    COUNT(DISTINCT customer_key)           AS nb_clients_uniques,
```

```

COUNT(DISTINCT channel_key)      AS nb_canaux,
MIN(exposure_date)                AS premiere_exposition,
MAX(exposure_date)                AS derniere_exposition
FROM fact_promo_exposure
GROUP BY 1
ORDER BY nb_expositions DESC;

```

5.2 — Couverture par canal

```

-- Quelle part des expositions vient de chaque canal ?
SELECT
    dc.channel_name,
    COUNT(*)                      AS nb_expositions,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_total
FROM fact_promo_exposure e
JOIN dim_channel dc ON dc.channel_key = e.channel_key
GROUP BY 1
ORDER BY nb_expositions DESC;

```

5.3 — Requête inverse : clients NON exposés

```

-- Combien de clients actifs n'ont pas été exposés à une campagne ?
SELECT
    COUNT(*) AS clients_actifs_non_exposes
FROM dim_customer c
WHERE c.is_current = TRUE
    AND c.customer_key NOT IN (
        SELECT DISTINCT customer_key FROM fact_promo_exposure
    );

```

Dans le brief : comparez `clients_actifs_non_exposes` au nombre total de clients actifs. Quelle fraction de la clientèle n'a pas été atteinte ?

5.4 — Livrable

Sauvegardez dans `sql/fact-types/s09-factless.sql` : requête couverture + canal + inverse. Squelette disponible.

Section 6 — Brief exécutif et arbre de décision

6.1 — Complétez l'arbre de décision

Le fichier `docs/fact-type-decision-tree.md` est déjà dans votre repo avec un tableau pré-rempli des processus NexaMart. Complétez les colonnes **Type** et **Justification** pour chaque processus.

Processus NexaMart	Table	Type de fait	Question CEO à laquelle il répond
Ventes	<code>fact_sales</code>	(à compléter)	(à compléter)
Ordres	<code>fact_orders_transaction</code>	(à compléter)	(à compléter)
Stock quotidien	<code>fact_daily_inventory</code>	(à compléter)	(à compléter)
Pipeline commandes	<code>fact_order_pipeline</code>	(à compléter)	(à compléter)
Exposition promo	<code>fact_promo_exposure</code>	(à compléter)	(à compléter)

L'arbre de décision général (à inclure dans votre document) :

Le processus enregistre un événement ponctuel ?
 OUI → Transaction Fact Table

Le processus capture un état à intervalles réguliers ?
 OUI → Periodic Snapshot Fact Table

Le processus suit un cycle de vie avec des étapes séquentielles ?
 OUI → Accumulating Snapshot Fact Table

Le processus enregistre une présence ou couverture sans mesure numérique ?
 OUI → Factless Fact Table

6.2 — Brief exécutif : structure attendue

Fichier : `answers/S09_executive_brief.md`

Le brief doit répondre à la question CEO :

« Quels processus NexaMart sont transactionnels, quels sont des snapshots, et quels sont de simples présences ? »

Structure minimale attendue :

```
# S09 Executive Brief - Les quatre types de tables de faits

## Question du CEO
[Répétez la question]

## Réponse exécutive
[3-5 lignes - le board peut-il avoir une vue complète de NexaMart ?
Quel type répond à quelle question ? Avec un chiffre réel par type.]

## Décisions de modélisation
[Tableau des 5 processus : processus | table | type | justification]

## Preuve
[Un chiffre réel par table de faits :
- Transactions : revenue total ou marge brute
- Snapshot : stock moyen ou jours d'approvisionnement
- Pipeline : % livré et délai moyen
- Factless : nb expositions et % clients non atteints]

## Le piège de la semi-additivité
[Montrez le ratio SUM vs AVG pour une catégorie de produit]

## Prochaine recommandation
[Une décision CEO que votre modèle rend maintenant possible]
```

Section 7 — Checklist avant soumission

7.1 — Artefacts requis

<code>answers/S09_executive_brief.md</code>	← brief CEO avec chiffres réels
<code>docs/fact-type-decision-tree.md</code>	← tableau complété (5 processus)
<code>sql/fact-types/s09-transaction.sql</code>	← requêtes CEO + vérification grain
<code>sql/fact-types/s09-periodic-snapshot.sql</code>	← SUM faux + AVG correct + requête CEO
<code>sql/fact-types/s09-accumulating.sql</code>	← funnel + délais
<code>sql/fact-types/s09-factless.sql</code>	← couverture + inverse NOT EXISTS
<code>db/nexamart.duckdb</code>	← base avec les 4 nouvelles tables chargées
<code>ai-usage.md</code>	← interactions significatives tracées

7.2 — Vérification finale

Vérifier que les 4 nouvelles tables sont bien chargées

```
duckdb db/nexamart.duckdb -c "  
SELECT table_name, estimated_size AS rows  
FROM duckdb_tables()  
WHERE table_name IN (  
    'fact_orders_transaction',  
    'fact_daily_inventory',  
    'fact_order_pipeline',  
    'fact_promo_exposure'  
);  
"
```

Lancer les checks automatiques

```
make check
```

Résultat attendu de `make check` : tous les checks PASS, aucun FAIL.

7.3 — Git push

```
git add -A  
git commit -m "S09: 4 types de tables de faits - transaction, snapshot, accumulating, factless"  
git push
```

Rappel : Remettez votre travail avant 18h00 le 15 juin — la séance S10 (Comité du board, non noté) commence le soir même.

Rubrique de notation

Dimension	Poids	Ce qui est évalué
Qualité du modèle	40 %	4 types correctement implémentés et documentés. <code>fact_daily_inventory</code> utilise snapshot, pas calcul.
Qualité de la validation	25 %	Requête snapshot retourne un résultat par période sans cumul des transactions. Démonstration SUM vs AVG présente.
Justification exécutive	20 %	Brief CEO avec chiffres réels. Arbre de décision complété et applicable à de nouveaux processus.
Traçabilité du processus	10 %	<code>docs/fact-type-decision-tree.md</code> documenté : quel type répond à quel type de question business.
Reproductibilité	5 %	<code>make check</code> passe sans erreur. Tous les fichiers SQL dans les bons chemins.